

# Git & GitHub Practical — Step-by-Step Lab

## Practical Project

We will create a simple project:

```
robot-project/  
├── robot.cpp  
└── README.md
```

Students will learn:

**Create** → **Initialize** → **Track** → **Stage** → **Commit** → **GitHub** → **Push** → **Modify** → **Push again** → **Branch** → **Merge**

---

## STEP 1 — Check Git

Open Terminal.

Type:

**git --version**

### What are we doing?

We are asking the computer:

"Do you have Git installed?"

If we get:

git version 2.x.x

Git is ready.

Or else

**sudo apt update**

**sudo apt install git -y**

---

## STEP 2 — Tell Git Who We Are

Type:

```
git config --global user.name "Your Name"
```

Example:

```
git config --global user.name "Rohith"
```

Then:cd

```
git config --global user.email "arun@example.com"
```

### What is happening?

Git needs to know:

Who made this change?

So every time we save a version, Git can record:

Changed by: Rohith

Check configuration:

```
git config --list
```

---

## STEP 3 — Create Our Robot Project

Type:

**mkdir robot-project**

Then:

**cd robot-project**

---

## STEP 4 — Create Robot Code

Create:

**nano robot.cpp**

Enter:

**#include <iostream>**

**using namespace std;**

**int main()**

**{**

**int battery = 90;**

**cout << "Robot is running" << endl;**

**cout << "Battery: " << battery << "%" << endl;**

**return 0;**

**}**

Save it.

## What happened?

We created our first robot program.

Our folder now looks like:

robot-project/

└─ robot.cpp

But Git doesn't know about the file yet.

---

# STEP 5 — Run the Robot Program

Compile:

**g++ robot.cpp -o robot**

Run:

**./robot**

Output:

Robot is running

Battery: 90%

## What happened?

We checked that our code actually works.

This is important:

**First make sure your project works, then save it with Git.**

---

# STEP 6 — Start Git

Type:

**git init**

To change default branch name from master to main:

**git branch -m main**

Output may look like:

Initialized empty Git repository

Now Git starts tracking this project.

To check whether git has initialised, check by using: **ls -la**

## What happened?

Git started watching this project.

Before:

robot-project/

└─ robot.cpp

After:

robot-project/

└─ robot.cpp

└─ .git/

The **.git** folder contains Git's information about the project.

## Analogy

Imagine hiring a security guard.

Before:

House

└─ Robot project

After:

House

└─ Robot project

↑

Git is watching

**git init** means: "Git, start watching this project."

---

## STEP 7 — Ask Git What's Happening

Type:

**git status**

You may see:

Untracked files:

robot.cpp

### What does "untracked" mean?

Git knows that the file exists, but it isn't keeping track of it yet.

Think:

robot.cpp

↓

Git sees it



"But you haven't told me to remember it."

---

## STEP 8 — Add the File

Type:

**git add robot.cpp**

For adding specific file,

**git add \***

Adds all non hidden files in the current directory to the staging area

**git add .**

Adds all hidden files, deleted files too unless ignored by .gitignore to the staging area.

Then:

**git status**

Now Git will show that **robot.cpp** is ready to be committed.

To unstage a file,

**git rm --cached robot.cpp**

For all files,

**git rm --cached -r .**

### What happened?

We told Git:

"I want you to remember this file in the next saved version."

The file moves conceptually:

### Working Area



**git add**



**Staging Area**

---

## 9. What is the Staging Area?

This is one of the most important concepts for children.

Imagine packing a school bag.

Your room has:

Book

Pencil

Lunch box

Toy

Water bottle

But you only want:

Book

Pencil

Water bottle

You put those into your school bag first.

That's like **staging**.

Your project



Choose what to save





Staging Area

Staging means "choose what I want to save next."

---

## STEP 10 — Commit

Now type:

```
git commit -m "Add first robot program"
```

For committing particular file,

```
git commit robot.cpp -m "Update robot program"
```

If you want a particular committed file back to staging

```
git reset --soft HEAD~1
```

A — B — C

↑

HEAD

- HEAD → current commit C
- HEAD~1 → previous commit B
- HEAD~2 → two commits before A

To move the commit back and remove the changes from your Working Directory and Staging Area.

```
git reset --hard HEAD~1
```

If there is no parent commit, then to remove the only version type,

```
git update-ref -d HEAD
```

**What happened?**

Git took the staged files and created a **saved version**.

### **Working Directory**

↓

**git add**

↓

### **Staging Area**

↓

**git commit**

↓

### **Local Repository**

↓

**git push**

↓

### **GitHub**

---

## **STEP 11 — See Our Saved Version**

Type:

**git log --oneline**

You may see:

a82f31c Add first robot program

### **What happened?**

Git is showing us the project's history.

Version 1

↓

Add first robot program

Later we'll have:

Version 1 → Version 2 → Version 3

---

## STEP 12 — Create README

Create:

**nano README.md**

Write:

**# Robot Project**

**##Sensors**

**##Ultrasonic Sensor**

**# → Heading 1 (main title) ## → Heading 2 ### → Heading 3 #### → Heading 4**

Save it.

### What is README?

README is like the **front page of a project notebook**.

It tells other people:

- What is this project?
- What does it do?

- How can I use it?
- 

## STEP 13 — Check Git Again

**git status**

Git says:

README.md

is untracked.

### Why?

Because we created a new file after our previous commit.

Git is basically saying:

"Hey! You created something new."

---

## STEP 14 — Add README

**git add README.md**

Or:

**git add .**

### What does **.** mean?

The dot means:

**Everything in this current folder.**

So:

**git add .**

means:

"Stage all changes here."

---

## STEP 15 — Commit Again

**git commit -m "Add project README"**

Now our history is:

- Add project README

|

- Add first robot program

### What happened?

We now have **two snapshots**.

---

## STEP 16 — Create GitHub Repository

Now go to GitHub.

Create:

**robot-project**

### What happened?

Until now, everything was on our computer:

YOUR COMPUTER

└─ robot-project

Now we're creating an online home:

GITHUB

└─ robot-project

---

## STEP 17 — Connect Computer to GitHub

GitHub gives you a repository address.

Then:

```
git remote add origin https://github.com/USERNAME/robot-project.git
```

### What does this mean?

We are telling Git:

"Here is the online home of my project."

Your Computer

|

| origin

↓

GitHub

### What is **origin**?

**origin** is simply a nickname.

Instead of saying:

`https://github.com/USERNAME/robot-project.git`

every time, we can say:

`origin`

---

## STEP 18 — Check the Connection

Type:

**git remote -v**

You should see the GitHub repository.

## What are we checking?

We're asking:

"Git, where is my online project?"

Git answers:

GitHub → robot-project

---

# STEP 19 — Push

Now:

**git push -u origin main**

**This is VERY important.**

What is happening?

Your Computer

|

| PUSH

↓

GitHub

We are sending our saved commits to GitHub.

## Analogy

Imagine:

Your notebook



Take a photocopy



Put it in online classroom

That's **push**.

**Remember:**

**PUSH = Send my saved work to GitHub.**

---

## STEP 20 — Look at GitHub

Refresh the GitHub page.

You should see:

robot-project

|— robot.cpp

|— README.md

🎉 Your project is now online.

---

## STEP 21 — Make a Change

Now change:

**int battery = 90;**

to:

**int battery = 80;**



## What happened?

Our working project changed.

Git notices:

Old:

90%

New:

80%

---

## STEP 22 — See the Change

Type:

**git diff**

Git may show:

- int battery = 90;

+ int battery = 80;

### What does this mean?

- = removed/old

+ = added/new

---

## STEP 23 — Save the New Version

First:

**git add .**

Then:

**git commit -m "Update battery value"**

Now Git has:

VERSION 2

↓

Battery = 80%

VERSION 1

↓

Battery = 90%

**This is Git's superpower.**

We can keep multiple project snapshots.

---

## STEP 24 — Push the New Version

**git push**

Now:

Computer

↓

push

↓

GitHub

GitHub now has the latest version.

If we change file in github and local,

## Step 1 — Download information about GitHub

Run:

**git fetch origin**

What does fetch do?

It contacts GitHub and asks:

"What is the latest state of main on GitHub?"

It does not change your files.

Think of it as:

GitHub

|

| git fetch

↓

Your computer learns:

"Hey, GitHub has a newer commit."

## Step 2 — Check the situation

Now run:

**git status**

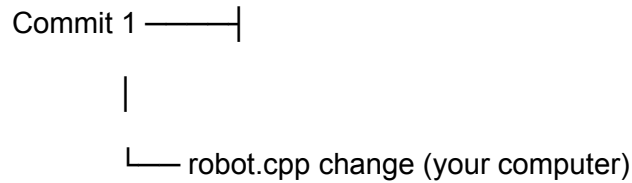
You will probably see something like:

Your branch and 'origin/main' have diverged

This means:

└── README change (GitHub)

|



Your local and GitHub histories have gone in different directions.

### Step 3 — Combine them

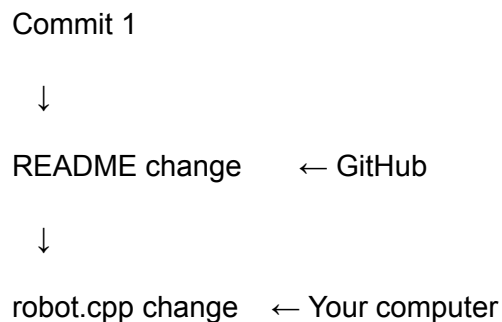
Now run:

**git pull --rebase origin main**

This tells Git:

"Bring the GitHub changes into my local repository, then put my local commit after them."

The history should become:



### Step 4 — Very important: watch the result

If you get:

Successfully rebased and updated refs/heads/main.

Great. Then run:

**git push**

else solve the issue by removing conflict

---

## STEP 25 — Introduce Branches

Tell the children:

"Our robot works. Now we want to add a LIDAR feature."

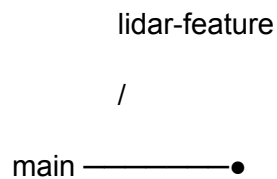
But we don't want to risk breaking our working robot.

So we create a **branch**.

**git switch -c lidar-feature**

### What happened?

We created a separate workspace for the new feature.



**Branch = A separate path for experimenting.**

---

## STEP 26 — Add LIDAR Feature

Modify the code:

```
int lidarDistance = 45;
```

```
cout << "LIDAR Distance: "
```

```
<< lidarDistance << " cm" << endl;
```

Test it.

Then:

**git add .**

And:

```
git commit -m "Add LIDAR feature"
```

## What happened?

We saved the LIDAR work **inside the branch**.

main

|

•

|

•————•

lidar-feature

The main project hasn't received the LIDAR change yet.

---

## STEP 27 — Merge

When we're happy with the LIDAR feature:

First go to main:

```
git switch main
```

Then:

**git merge lidar-feature**

## **What happened?**

We took the work from:

lidar-feature

and combined it with:

main

Think:

Experiment

↓

Test

↓

Works!

↓

Merge into main

---

## **STEP 28 — Push the Final Project**

**git push**

Now GitHub gets the merged version.

---

## **STEP 29 — Move back to previous versions**

Now if we want to see and choose previous versions in local repository,

**git log --oneline**

You'll see previous versions with address,

To select that version,

**git restore --source=626ddf2 robot.cpp**

Now your working file contains the old version.

Check it:

**vi robot.cpp**

---

## STEP 29 — Clone vs fetch vs pull

### 1. git clone

Use clone when you don't have the project yet.

**git clone https://github.com/user/robot-project.git**

**What happens?**

GitHub

|

| clone

↓

Your Computer

Git downloads the repository and creates a local copy.



For example:

GitHub:

robot-project

├── robot.cpp

├── lidar.cpp

└── README.md

After cloning:

Your Computer:

robot-project

├── robot.cpp

├── lidar.cpp

└── README.md

---

## 2. git fetch

Use **fetch** when you already have the project but want to check what has changed on **GitHub**.

**git fetch**

**What happens?**

GitHub

|

| fetch

↓

Your Computer

But importantly, **your working files are not changed.**

Suppose GitHub has:

robot.cpp

lidar.cpp

camera.cpp

Your computer has:

robot.cpp

lidar.cpp

You run:

git fetch

Git learns about the new `camera.cpp` and updates its remote-tracking information, but your current working project doesn't suddenly get `camera.cpp`.

---

### 3. `git pull`

Use **pull** when you want to get the latest changes and integrate them into your current branch.

git pull

Conceptually:

git pull

=

git fetch

+

integrate the changes

So:

GitHub

|

| pull

↓

Your Computer

↓

Current project gets updated

**Pull = "Give me the latest changes and update my project."**

---

## 4. The Main Difference

| Command                | When?                        | What happens to your project?                              |
|------------------------|------------------------------|------------------------------------------------------------|
| <code>git clone</code> | First time getting a project | Creates a complete local copy                              |
| <code>git fetch</code> | Check for remote updates     | Downloads remote information, doesn't update working files |
| <code>git pull</code>  | Get remote updates           | Fetches + integrates changes                               |

## The Complete Story

Draw this on the board:

GITHUB

↑

|

PUSH

↑

COMMIT

↑

ADD

↑

WRITE CODE

|

↓

EDIT

And explain each one:

| Command | Child-friendly meaning |
|---------|------------------------|
|---------|------------------------|

|                       |                    |
|-----------------------|--------------------|
| <code>git init</code> | Start Git watching |
|-----------------------|--------------------|

|                         |                    |
|-------------------------|--------------------|
| <code>git status</code> | What is happening? |
|-------------------------|--------------------|

|                      |                     |
|----------------------|---------------------|
| <code>git add</code> | Choose what to save |
|----------------------|---------------------|

|                         |                       |
|-------------------------|-----------------------|
| <code>git commit</code> | Take a snapshot       |
| <code>git log</code>    | See old snapshots     |
| <code>git push</code>   | Send to GitHub        |
| <code>git pull</code>   | Get latest work       |
| <code>git clone</code>  | Copy a GitHub project |
| <code>git branch</code> | Make a separate path  |
| <code>git merge</code>  | Combine the paths     |
| <code>git diff</code>   | See what changed      |

---

## The 5 Commands Children Must Remember

Don't make children memorize 30 commands initially.

Start with these:

## 1. Check

**git status**

"What's happening?"

## 2. Select

**git add .**

"I want to save these changes."

## 3. Save

**git commit -m "message"**

"Take a snapshot."

## 4. Upload

**git push**

"Send my saved work to GitHub."

## 5. Download

**git pull**

"Get the latest work from GitHub."

---

# The Children's Memory Trick

Tell them to remember:

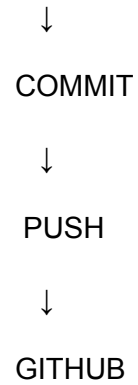
WRITE



STATUS



ADD



**In words:**

**Write → Check → Choose → Save → Send**

And:

**Git = Time Machine**

**GitHub = Online Home**

**Add = Pack your bag**

**Commit = Take a photograph**

**Push = Send it online**

**Pull = Bring updates back**

**Branch = Make a copy to experiment**

**Merge = Combine the experiment with the main project**